

만들어진 것과 커진 것은 다릅니다

제가 「구현하라」고 지시한 것 셋이 전부 이미 있었습니다. 역할별 화면 틀도, 모니터링도, 백업 자동화도 코드는 다 있었고 켜져 있지 않았을 뿐이었습니다. 없던 것은 배선과 주기였습니다. 그리고 사용성 20% → 55%를 올린 것은 새 코드가 아니라 배선이었습니다.

D-377 · 초록의 착시 ⑨ — 배선의 착시

시험은 함수를 직접 불러 통과시키고
운영은 그 함수를 부르지 않습니다

둘 다 초록입니다. ⑥(스키마)이 「필드가 있으면 기능이 있다」였다면, ⑨(배선)은 「코드가 있으면 동작한다」입니다 — 한 단계 위입니다. ★ 이미 만든 것을 켜는 것이 새로 만드는 것보다 쉽습니다. 오늘 세 번 증명했습니다.

67.3%

상용 진행율 (+7.8%p)

20 → 55%

사용성 · 배선이 올림

300 = 300

폭주 투입 = 실행 · 버림 0

v3.6 → v3.7

187 → 195

67.3% — 사용성이 35%p 뛰었습니다

상용 출시

67.3%

8영역 가중 추정 · 직전 59.6%
+7.8%p 단위 시험 530

계약 이행

82%

절 32/39 실속
상대방 대기 6절 한 줄로만 보고

스크린샷

0/16

화면은 코드에 썼으나
빌드 환경 어긋남 3쪽 판정

영역	가중	완성율	이번 턴	남은 잡재	근거
① 기능 완결성	20%	83%	+1%p	+3.4%p	이벤트 큐잉 전환
② 보안	20%	68%	+3%p	+6.4%p	쓰기 면 47자리 전수 · 익명 실사용 4자리 차단
③ 격리·다중테넌시	10%	88%	—	+1.2%p	유지
④ 운영	15%	60%	+15%p	+6.0%p	모니터링·백업 주기 연결 — 코드는 이미 있었음
⑤ 성능·확장	10%	45%	+10%p	+5.5%p	이벤트 행 950B · 추론 제외 상한 51.3건/초
⑥ 품질	10%	84%	+2%p	+1.6%p	530 passed · 검증 34종
⑦ 사용성	10%	55%	+35%p	+4.5%p	K3 배선(역할 15종 → 7종 이음) + 화면 셋 + 32칸 부작위 시험
⑧ 법·인증	5%	18%	—	+4.1%p	미착수

★ 두 턴 동안 안 움직이던 사용성이 20% → 55%로 뛰었습니다. 그런데 새 코드가 아니라 배선이 올랐습니다. ④ 운영의 +15%p도 마찬가지입니다 — 모니터링과 백업은 코드가 이미 있었고 「주기」만 없었습니다. 이번 턴 +7.8%p 중 절반 이상이 「이미 있던 것을 켜 결과」입니다.

잠자는 기능을 전수합니다

제 지시서	실측
K3 프레임 「구현하라」	이미 서 있었다. 없던 건 배선 — 설정이 비어 모두가 같은 화면을 봤다
모니터링 3종 「구현하라」	ops_monitor.py가 이미 있었다. 없던 건 주기
백업 자동화 「구현하라」	ops_backup/restore.py가 이미 있었다. 없던 건 주기

없던 것은 코드가 아니라 「배선」과 「주기」였습니다.
★ 이번 인수 자산의 성질입니다. 전 개발자가 만들어 놓고 켜지 않았습니

§1 술어 셋 — 한 수로 합치지 마십시오

고치는 방법이 다릅니다

- ㉠ 호출 없음

고치는 법 · 배선

정의는 있으나 운영 경로(뷰·태스크·시그널·커맨드)에서 도달 불가. 시험은 함수를 직접 불러 통과시킵니다.
→ 목록만 내고 나에게 보고하라. 배선은 판단이 필요합니다
- ㉡ 주기 없음

고치는 법 · 등록

주기적으로 돌아야 하는데 스케줄러에 등록되지 않음. 모니터링·백업이 이 갈래였습니다.
→ 이번 턴에 켜라. 등록이라 씁니다
- ㉢ 설정 빔

고치는 법 · 데이터

코드는 분기하는데 설정·표가 비어 항상 같은 가치로 갑니다. K3가 이 갈래였고, 모두가 같은 화면을 봤습니다.
→ 이번 턴에 켜라. 데이터라 씁니다

도구 verify_dormant.py

D-301 건수 출력

세 술어별 각각 [실측] 건수를 냅니다. 저는 그 수를 모릅니다 — 실측으로 답하십시오.
기본본은 래치(D-311). ★ 새로 만드는 것은 「켜진 상태로 태어나야」 합니다 — 운영 경로 연결 없이 커밋되면 exit 1.

★ 켜 때마다 그 기능이 실제로 도는 것을 시험이 봅니다. 켜기만 하고 안 도는 것은 착시 ㉠의 재발입니다.

§2 착시 — 이제 아홉 형입니다

형	이름	무엇을 대신 답하나
① ~ ⑤	부착물 · 시나리오 수 · 모수와 술어 · 탐지기 자체 · 환경	기존
⑥	스키마	필드가 있으면 기능이 있다고 읽는다
⑦	캐시	고침이 반영되기 전에 고쳤다고 읽는다
⑧	봉투	겉의 200이 안의 403을 덮는다
⑨	배선 (신설)	코드가 있으면 동작한다고 읽는다 — ⑥보다 한 단계 위

문을 여는 것은 「빌드 1회」입니다

목표: 타입오류 3,229건 → 0

목표: 빌드 1회 성공 → 스크린샷 3장

0/16 → 3/16

먼저 문턱을 낮춥니다. 스크린샷에 필요한 것은 타입이 깨끗한 것이 아니라 화면이 뜨는 것입니다. 타입오류가 3,229건 있어도 빌드는 될 수 있습니다 — 타입검사와 번들링은 다른 일입니다.

지시: 「빌드가 되는가」와 「타입이 깨끗한가」를 분리해서 답하십시오. 빌드 1회만 성공하면 0/16이 3/16이 됩니다. 타입은 그 다음입니다.

§ 판 결정 — 「우리가 못 정한다」를 그대로 두지 않습니다

- 1 저장소의 package.json·lock이 요구하는 판 **실측**
- 2 컨테이너에 설치된 판 **실측**
- 3 둘이 다르면 — 저장소가 정본입니다. 컨테이너를 저장소에 맞춥니다. 그 반대가 아닙니다
★ 이유: 저장소는 우리가 커밋한 것이고 컨테이너는 누군가 만든 이미지입니다. 버전의 진실은 lock 파일에 있습니다
- 4 lock이 없거나 어긋나면 그것이 진짜 결함입니다. lock 고정이 첫 일입니다

★ 타입오류 3,229건 중 새 화면 셋은 0건 — 이 사실이 처리를 정합니다. 우리 코드는 깨끗하고 기존 코드가 어긋납니다. 전면 수정 금지(D-358 계열) — 새 코드 100% · 기존 래치. 3,229를 0으로 만드는 일은 이번 국면의 일이 아닙니다.

FRONTEND_DEPS_DRIFT를 대장에 올린 것 인정. unlock_type은 DEV입니다 — 이관 자산이지만 판을 맞추는 일은 우리가 합니다. CONTRACT가 아닙니다.

§ D-382 · AI 자격증명 — 막은 것을 유지하되, 여는 절차를 함께

판정: 막은 것을 되돌리지 마십시오. 계약 경로(gRPC · F-02·F-10)에 안 닿는다는 실측이 있으니 계약 위험이 없고, 인증 없는 콜백을 열어 두는 쪽이 더 나쁩니다.

★ 그러나 막기만 하고 여는 절차가 없으면 다음 배포에서 조용히 깨집니다. 함께 만드십시오 — 자격증명 저장처(표 ②)에 등재 · inbound_접두(AI 서버가 우리를 부르는 키) · 배포 전 점검표에 「이 키가 present 미만이면 배포하지 않는다」를 게이트로.

원칙: 막은 것에는 여는 절차가 함께 있어야 합니다. 차단은 절반이고, 나머지 절반은 복구 경로입니다.

§ D-383 · 역할 ↔ 프리셋 7종 — 승인. 나머지 8종은 고객이 정합니다

7종 이름 승인합니다. 「인용이 아니라 우리 판정」이라고 밝힌 것이 정확하니 그 표기를 대장에 남기십시오. 나머지 8종은 「미배선」으로 등재하고 추측으로 채우지 마십시오.

★ 이 매핑의 정답은 우리에게 없습니다. 역할별로 무엇이 보여야 하는지는 안양 관제팀이 압니다. → 검수 콘솔에 「역할 ↔ 화면 매핑 확인」 항목을 추가하고, 우리가 이은 7종을 보여 주며 「맞는가 / 빠졌는가 / 이진 보이면 안 된다」를 받습니다.

★ 이것이 검수 콘솔의 첫 실전 용도입니다 — 화면이 없어도 할 수 있는 검수입니다.

제 판정은 반쪽 틀렸고, 지시는 세 번 틀렸습니다

"무효화를 버리면 낡은 화면이 남고, 폭주는 곧 재난이니
그때 관제 화면이 옛 데이터를 보여 줍니다 — 보는 사람에겐 같은 일입니다."

제가 틀린 부분: 이벤트는 사라지지 않았습니니다. @rate_limited(200)은 캐시 보호 파일 하나뿐이고, 이벤트 행은 record_detection이 트랜잭션 안에서 저장하며 **상한도 큐도 없습니다**. 「재난 때 이벤트가 사라진다」는 일어나지 않았습니니다 — 제 판정은 **원인을 잘못 짚었습니다**.
★ Code가 맞은 부분, 그리고 이것이 더 중요합니다: **결함의 등급은 내부 원인이 아니라 「사용자에게 무엇으로 보이는가」로 정합니다**. 이벤트가 사라지는 것과 화면이 안 바뀌는 것은 내부적으로 전혀 다르지만 **당직자에게는 같습니다**.
처리도 옳았습니다 — 큐잉 전환 · **같은 (모델,pk) 접기**(그 접기가 상한이 하던 부하 억제에 대신합니다) · **심각은 큐가 가득 차도 그 자리에서 처리 · 투입 300 = 실행 300 · 버린 것 0건**.

§ D-379 · 「구현하라」를 쓸 자격이 저에게 없습니다

제가 D-369에서 「구현을 지시하기 전에 이미 있는지 실측한다」를 원칙으로 세웠고, 그 원칙이 실린 지시서 안에서 세 건이 이미 있었습니다. 원칙을 적는 것으로는 부족합니다.
★ 그래서 앞으로 저는 「구현하라」를 쓰지 않습니다. 「이 기능이 동작하게 하라」로 씁니다. 만들지 겠지는 Code가 실측해서 정합니다.
이건 D-312(보안 지시는 수단이 아니라 금지 대상으로)의 형제입니다 — **수단을 지시하지 않습니다**.
집행: 제 지시서의 「동작하게 하라」 항목은 착수 전 **한 줄 회신** — 이미 있음(경로) / 부분 있음(무엇이 없나) / 없음. 이번처럼 넷을 한 표로 낸 것이 정본 형식입니다.

§ D-380 · 결정 번호는 저장소가 정본입니다

제 D-367-368이 Code 트리의 기존 번호와 충돌했고, **뺐지 않고 D-369~376으로 잇고 대응표를 남긴 것 — 정확합니다**. 뺐었으면 두 이력이 어긋났을 것입니다.
규약: 제 지시서의 번호는 제안입니다. 착수 시 저장소 기준 다음 가용 번호로 재배치하고 대응표를 남깁니다. 매 턴 상시 규약으로 두니 다시 묻지 마십시오. 이 지시서는 그 규약대로 **D-377부터 시작합니다**.

쓰기 면 20 → 47자리 정정도 인정합니다. 근거가 대장에 없어 인용하지 않고 다시 센 것이 D-322의 정확한 집행입니다 — 제가 만든 규약이 제 숫자를 거부했습니다. 익명이 실제로 쓰던 4자리는 넷 다 막혔고 \$0.4 밖이었습니다.

실행 순서

- 1 D-377~384 정서 — v3.6/187 → v3.7/195 · 저장소 기준 번호로 재배치 + 대응표 D-380
- 2 ★★ 잠자는 기능 전수 — 세 술어별 **실측** 건수 · ㉠부터 켜기 · 켜는 것이 도는 것을 시험이 본다 · ㉡은 목록만 내고 보고 D-377 · 첫 항목
- 3 ★ 빌드 1회 성공 — lock 실측 → 컨테이너를 저장소에 맞춤 → 빌드. 「빌드되는가」와 「타입이 깨끗한가」를 분리해 답하라 D-381
- 4 ★ 스크린샷 3장 — 화면 3개를 E2E 중 캡처 → 검수 콘솔 0/16 → 3/16 D-347
- 5 영상 백업 — MinIO 도달부터(달지 못하는 이유가 설정이면 그것도 ㉢) · 이벤트 묶음 · 실제 복구 · 보존 기간 D-384 ㉣
- 6 AI 자격증명 여는 절차 — 표 ㉤ 등재 · inbound_ 명명 · 배포 전 점검 게이트 D-382
- 7 역할 매핑 — 7중 「우리 판정」 표기 · 8중 「미배선」 등재 · 검수 콘솔에 확인 항목 추가 D-383
- 8 석 줄 보고 — 계약은 「상대방 대기 6절」 한 줄 · 상용 목표 +7절 · 지시 이행 D-353
- 9 여기서 멈춘다. 잠자는 기능이 몇 건이었는지와 스크린샷 몇 장이 컸는지 **실측**으로 보고하라 보고 후 판정

※ 하지 않는 것: 타입오류 3,229 정리 · 나머지 화면 13개 · 8중 매핑 추측 채우기.
 ※ 새 게이트는 ㉡의 `verify_dormant.py`와 ㉢의 배포 전 점검에만 허용합니다(D-353 비율 규정).

§ 대표께 — 오늘 배운 것 하나

제가 「구현하라」고 지시한 것 셋이 전부 이미 있었습니다. 역할별 화면 틀도, 모니터링도, 백업 자동화도 코드는 다 있었고 켜져 있지 않았을 뿐이었습
 니다.

★ 이견 인수 자산의 성질입니다. 전 개발자가 만들어 놓고 운영에 연결하지 않은 것입니다. 셋이 같은 모양이면 전체에 더 있다고 보는 것이 합리적이라,
 「잠자는 기능」 전수 조사를 이번 턴 첫 항목으로 올렸습니다.

★ 이미 만든 것을 켜는 것이 새로 만드는 것보다 쉽습니다. 오늘 그것이 세 번 증명됐습니다 — 사용성 35%p와 운영 15%p가 그렇게 올랐습니다.

제 판정 하나는 반쪽 틀렸습니다. 「재난 때 이벤트가 사라진다」는 일어나지 않았습니 — 버려지던 것은 이벤트가 아니라 화면 갱신 신호였습니다. 다
 만 Code의 판단이 옳았습니다: 폭주는 곧 재난이고, 그때 관제 화면이 옛 데이터를 보여 주면 당직자에게는 같은 일입니다. 결함으로 처리해 고쳤고 폭주
 300건에 버린 것 0건입니다.

화면 스크린샷은 아직 0장입니다. 화면은 코드에 썼는데 프론트엔드 빌드 환경이 어긋나 있습니다. 문턱을 낮췄습니다 — 타입오류 3,229건을 0으로 만
 들 필요는 없고, 빌드 1회만 성공하면 3장이 찍힙니다. 그 3,229건 중 우리가 새로 만든 화면 셋은 0건입니다.